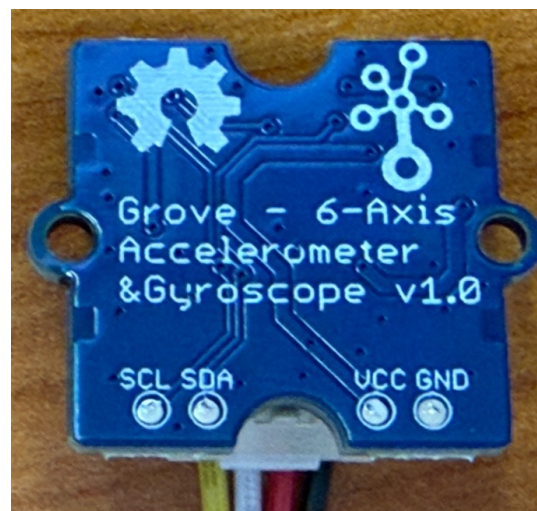
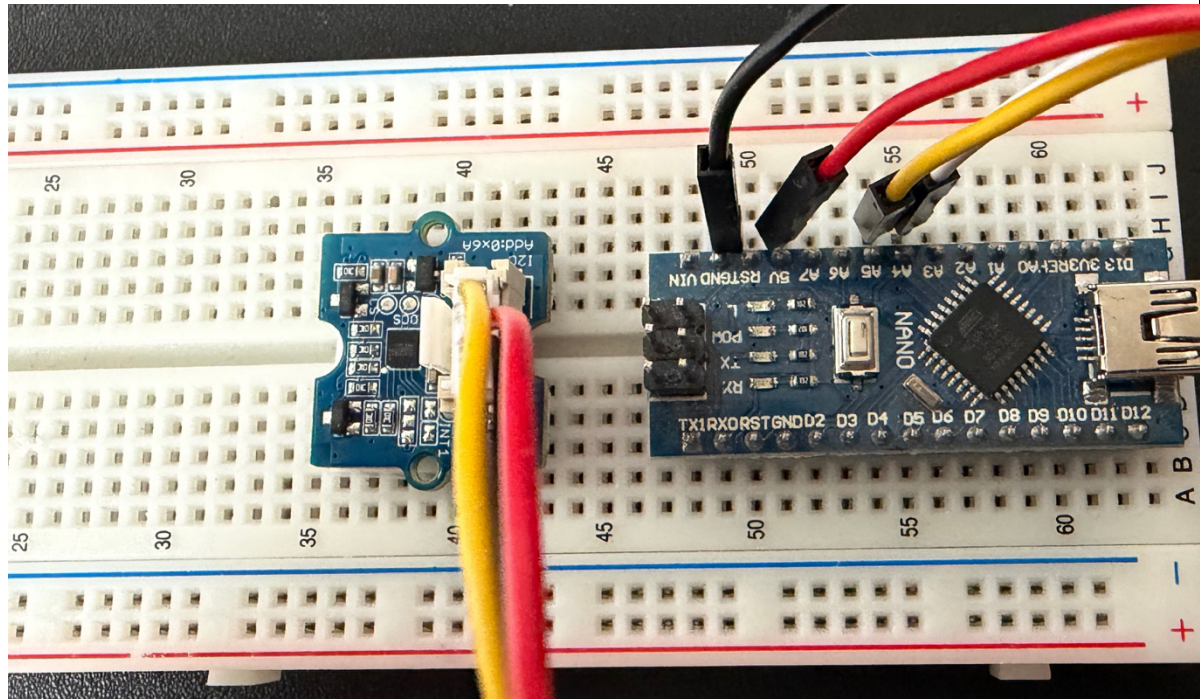


ID N3
Function and characteristics of an inertial measurement cluster STM LSM6DS3
Required Knowledge • Functionality of inertial measurement units (IMUs) • Sensor characteristics: Measurement range, resolution, accuracy, characteristic curve, sensor errors • I2C interface • Datasheet STM LSM6DS3 (https://content.arduino.cc/assets/st_imu_lsm6ds3_datasheet.pdf https://files.seeedstudio.com/wiki/Grove-6-Axis_AccelerometerAndGyroscope/res/LSM6DS3TR.pdf) • Documentation of the breakout board (https://wiki.seeedstudio.com/Grove-6-Axis_AccelerometerAndGyroscope/) • STMicroelectronics Application Notes (https://www.st.com/resource/en/application_note/an5130-lsm6ds3trc-alwayson-3d-accelerometer-and-3d-gyroscope-stmicroelectronics.pdf) • Programming an Arduino (Nano) with the Arduino IDE • Required library: "Seeed Arduino LSM6DS3" (Version 2.0.5) • Programming in Python (Anaconda/Spyder) • Python and python libraries (z.B. https://py-tutorial-de.readthedocs.io/de/python-3.3/ , https://www.python-kurs.eu/numpy.php , ...) • Software provided as a starting point (the software must be modified!) • Specifically: Review & understanding of the header file LSM6DS3.h and its content How do I set the measurement range? How do I switch between different filter- or bandwidth-settings? • Hands-on oscilloscope incl. probe calibration
Material • Arduino Nano • LSMDS3 breakout board • Breadboard • USB-Mini cable • 4 connectors • Laptop, Arduino-IDE, Python-IDE (Anaconda, Spyder) • Oscilloscope
Total time: 4 h

Hardware Setup

SCL
(yellow)

SDA
(white)

VCC
(red)

GND
(black)

Color coding of the cables:

RED: 5V(3,3V)

BLACK: GND

WHITE: SDA (Arduino Nano: A4)

YELLOW: SCL (Arduino Nano: A5)

Part 1 – Setup

- Setup the Arduino and the inertial sensor cluster on the breadboard
- Please make sure that you do not create a short circuit between the supply voltages 3.3V or 5V to GND!
- Place the setup in a stable position that is as vibration-free as possible
- If available use a tripod and mount the sensor in a defined and vibration-free position
- Four connections are required: 5V(3.3V), GND, SDA and SCL
- Download the program "LSM6DS2_lowlevel", compile it and transfer it to the Arduino
- Check the transmitted data with "Tools/Serial Monitor" or "Tools/Serial Plotter"; make sure that the transmission parameters (COM port and baud rate) are set correctly
- Download the "Basisprogramm_ReadUSB_IMU" program and load it into your editor under Anaconda/Spyder
- Start the program there and check the output values. You may have to install the "pyserial", "numpy" and "matplotlib" libraries in the Anaconda Navigator beforehand
- Set the baud rate in the "Tools/Serial Monitor" to different values and observe the output
- Before doing the next exercises: Make sure that the COM port and baud rate are also set consistently in the program
- Analyze both programs: Which measurement data is recorded and how does the measurement data get to the laptop?

Deliverables:

- Photo of your setup with all pins and connections visible
- Serial output (screen dump of Tools/Serial Monitor) for different matching/not-matching communication parameters
- Hardware block diagram and Data-Flow diagram including a short, written explanation in your own words.

Part 2 – Sensor Configuration

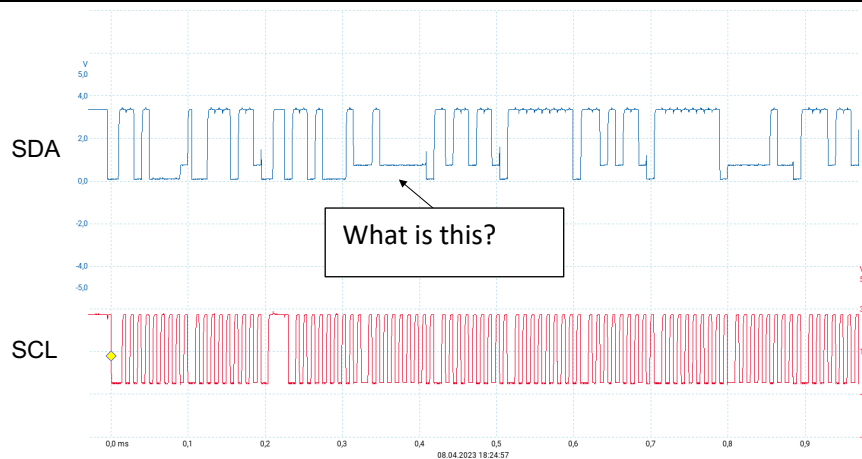
- Find out how to set the sensor cluster to different bandwidths, measurement ranges, and data rates by analyzing the data sheet, the application note, and the file "LSM6DS3.h" in the library folder (header file)
- Which registers have to be configured? Please review the Arduino-code provided.
- Which adjustments are required to change the measurement range of the z-axis of the acceleration sensor?
- Try out all configurations for the measuring range (2g, 4g, 8g, 16g) of the z-channel of the accelerometer and measure the digital output values for $a = -1\text{ g}$; 0 ; $+1\text{ g}$ (turn sensor element accordingly). Do the values make sense? Why?
- What is the resolution of each of these measurements?

Deliverables:

- Histograms and table of results for all 4 measurement ranges for 1 axis of the sensor including the output value for $+1\text{g}$, 0 , -1g (use gravity as the reference)
- Explanation – how do the values measured relate to the measurement range selected? What is the respective resolution in g and in m/s^2 ?

Part 3 – Oscilloscope measurements on the I2C Bus

- Connect the oscilloscope to your measurement setup. Measure the voltage between SCL and GND with a probe (perform a probe calibration first) and examine the data line SDA with a second probe.



- Compare your measurement result with the I2C data protocol from the lecture (or e.g. from [https://en.wikipedia.org/wiki/I²C](https://en.wikipedia.org/wiki/I%C2%B2C)).

Deliverables:

- Photo of the setup
- Screen dump of your measurement
- Analyze one telegram and completely annotate the respective bits in the telegram. Which deviations do you find?
- Answers to the following questions:
What is the data-rate? How many bits (raw) are transferred per second?
Analyze and annotate a single I2C telegram based on your oscilloscope measurement.
How does your measurement compare to the physical layer of the ideal I2C?

Part 4 – Noise on an acceleration sensor (measurement range: $\pm 2g$)

- Analyze the noise performance of one of the three axes of the accelerometer by configuring different Output Data Rates (ODRs) and observing the sensor output without any movement of the sensor; ODRs to be analyzed are 13Hz, 104Hz, 833Hz, 3330Hz, and 6660Hz
- To do this, keep the sensor vibration-free/still and carry out a measurement of at least 1000 values. Check the measurement in the time domain for outliers, filter them out if necessary, using a suitable filter (either on the Arduino or on your computer).
- If applicable: Repeat the measurement including your software-filter
- Compare the time-series, the histograms, mean values and standard deviations for different ODR settings and before/after implementation of your software filter

Deliverables:

- Histograms, Time Series, Mean/Median/StandardDeviation for each of your chosen settings and with/without filter
- Compare the histograms for all chosen settings and, if applicable, with or w/o filter. Document both mean value, standard deviation and include a plot StandardDeviation(ODR Setting). What are the reasons for the differences?
- Graph: standard deviation (ODR-setting)
- Calculate the sensor offset for each measurement. Do you notice any unexpected characteristics?
- Document your filter and include source code and explanation of the function into your report.

Part 5 –Noise on a yaw rate sensor (measurement ranges 245dps and 500dps)

- Analyze the noise performance of one of the three axes of the gyroscope for the different bandwidths available (13Hz, 104Hz, 416Hz, 1.66kHz).
- To do this, keep the sensor vibration-free/still and carry out a long-term measurement, at least 1000 values. Check the measurement in the time domain for outliers, filter them out if necessary, using a suitable filter (either on the Arduino or on your computer).
- Repeat the measurement including your software-filter

- Compare before/after implementation

Deliverables:

- Histograms, Time Series, Mean/Median/Standard Deviations for all settings and with/without filter
- Create and compare the histograms for all tested settings and, if applicable, with or w/o filter. Document both mean value, standard deviation and include a plot of the histogram. What are the reasons for the differences?
- Graph: standard deviation (ODR)
- Calculate a sensor offset and explain your calculation.
- Document your filter and include source code and explanation into your report.